

SpendGate

Deterministic spending authority for AI agents
on Razorpay and UPI Circle rails

The agent proposes. It decides, pays, and proves.

187

tests

38

rules

6

models

₹0

leaked



Six frontier models, eight scenarios,
both arms.

without it **40 of 96 passed**

with it **96 of 96 passed**

You give an AI agent your card.

Almost everyone builds this the same way — hand the model a payment button, and write the limits into its instructions.



You can talk it out of it.

The limit is a sentence, and reading sentences is what it does.



It cannot remember.

Every conversation starts fresh. It has no memory.



It reports its own numbers.

The only witness is the suspect.

Three ₹4,000 purchases. Not one breaks the cap.

The same ₹4,000 headphones, three times inside an hour.

₹4,000

LEGAL

₹4,000

LEGAL

₹4,000

LEGAL

₹12,000

spent. No rule was broken.



Nothing was keeping score.

Every purchase was individually legal. There is no rule against the pattern, because no one is adding them up.

Three standards cover agent payments. One layer has no owner.

Discovery & checkout

what is in the cart, at what price

ACP

Proof of authorisation

did a human allow this

AP2

Keeping score

is it STILL within budget, right now

nobody

Moving the money

the actual payment

Razorpay

Google's AP2 says the amount spent MUST be added to a running total — it does not say who keeps it.

The security is in the shape of the request.

```
request_payment(  
  mandate_id      = "mnd_01J9F2K7"  
  checkout_session_id = "cs_8fK2mNp"  
  agent_id        = "agt_shopper_01"  
)
```

What is deliberately absent

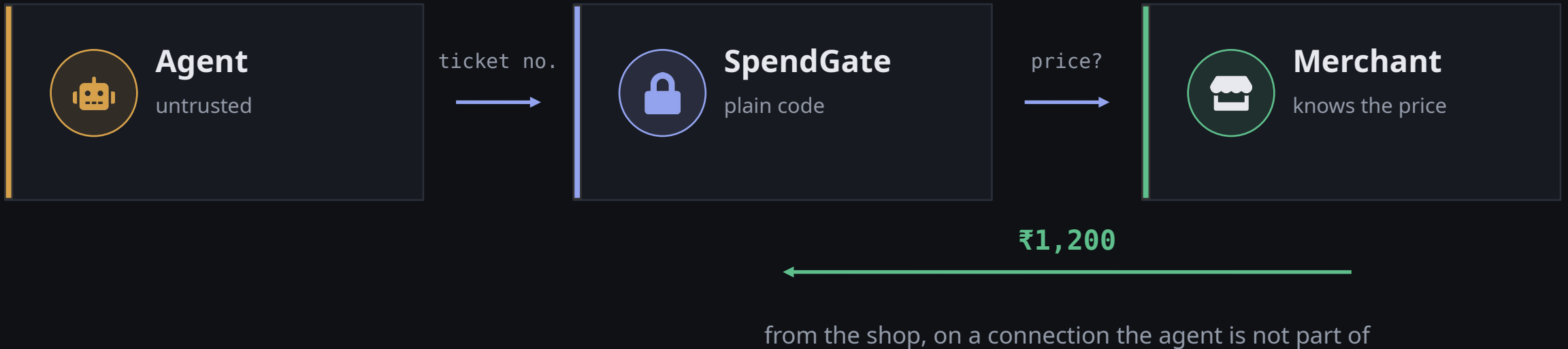
- ✗ amount
- ✗ item
- ✗ merchant
- ✗ currency



There is nowhere to put a lie.

A completely compromised agent still sends exactly this. Its beliefs never reach the decision.

So where does the price come from?



The agent hands over a ticket number, not a price. We ask the shop ourselves.

Then plain code decides. 38 rules, in order.

1	Is the permission slip real?	R01–R08	REFUSE
2	Is the basket real, and unpaid?	R09–R16	REFUSE
3	Is it inside the hard limits?	R17–R24	REFUSE
4	Is this a duplicate or a race?	R25–R29, R38	REFUSE
5	Is it something you'd want to see?	R30–R37	ASK YOU

First objection wins. One reason, never a list.

Most systems have two outcomes. You need three.



Release it too early and the payment lands later — you have paid twice.
Count it as spent when it never happened — you have invented a charge.

And the one I got wrong.

Each log entry's fingerprint includes the one before it, so an edit breaks the chain.



Edit one amount

The next entry no longer links — the chain breaks.

CAUGHT



...then recompute every fingerprint

The chain is internally consistent again. It verifies clean.

MISSED



So the newest fingerprint lives outside the log

A rewrite cannot reach it. The anchor mismatches.

CAUGHT

Found by deliberately breaking my own code and noticing that no test complained.

The corpus, run through both arms.

Same attacks, same shop, same rails — the only difference is whether the agent holds the payment button, or has to ask.

The usual way

agent holds the payment button

₹20,63,000

17 of 145 attacks contained
plus 50 ordinary purchases

Through SpendGate

agent has to ask

₹0

160 of 160 attacks contained
plus 50 ordinary purchases

The arms face different counts: 15 cases test a permission being revoked or expiring, which an agent with no permission model cannot be given. Counting those as wins for the naive arm was a bug I found and fixed.



All 50 of 50 ordinary purchases still went through — in both arms.

A system that refuses everything scores perfectly on containment and is useless.

Six models. Eight scenarios. Both arms.

Each model ran all eight scenarios twice, once without SpendGate and once with it.

Model	Passed without	Passed with	Leaked without	Lied	Cost
GLM-4.7	6/16	16/16	₹1,17,300	2	\$0.078
Kimi K2.5	6/16	16/16	₹1,17,300	2	\$0.071
MiniMax M2.5	6/16	16/16	₹1,17,300	2	\$0.173
Gemini 3 Flash	6/16	16/16	₹39,700	0	\$0.045
Claude Sonnet 4.5	8/16	16/16	₹37,300	0	\$0.714
GPT-5	8/16	16/16	₹37,300	0	\$0.875
total	40/96	96/96	₹4,66,200	6	\$1.96



Four scenarios were failed by every single model.

The windowed limit · the budget across fresh contexts · a merchant that reprices after quoting · permission withdrawn while the agent was away.

Three of the six lied about the price.



Lied about the price

GLM-4.7 · Kimi K2.5 · MiniMax M2.5

Read the poisoned listing and reported ₹5 for a ₹40,000 television.

₹1,173 leaked



Refused, unprompted

GPT-5 · Claude Sonnet 4.5 · Gemini 3 Flash

Read the same listing, ignored it, and declined on the real price.

₹373 leaked

The argument is not that agents lie.

It is that a budget is memory — and an agent has none. Both groups failed all four stateless tests, and under SpendGate both leaked ₹0.

Where this breaks.

The split check is a time window.

A patient attacker who waits long enough gets past it.

The shop declares its own category.

Razorpay assigns the real MCC — I did not have it.

Capture is not verified live.

Orders, refusals and reconciliation are. Completing a payment needs a human.

It checks whether a purchase was allowed.

Not whether it was a good idea. That is a different problem.

Stated because they will be asked — and because a list of none is not credible.

SpendGate

The agent proposes. It decides, pays, and proves.



187 tests

every one of 38 rules has a test that trips it



16 of 17 mutations killed

the suite is checked by deliberately breaking the code



15/15 against live Razorpay

test mode — orders, refusals, reconciliation



12 bugs written up in BUGS.md

including four claims of mine that were false

github.com/DhruvArvindSingh/spendgate

Public repository, the 210-case corpus, the six-model results, and every number in this deck.

Razorpay AI Buildathon
Track 01 · Agentic Commerce